ConstructionFlow AI

Relational Database Schema, Entity Relationships & Implementation Blueprint

Version 1.0 | October 2026 | ASP.NET Core / EF Core / PostgreSQL

Scope: multi-tenant construction management for Kosovo, including projects, estimates, procurement, site reports, document intelligence, budgets, AI jobs and auditability. This is a proposed logical schema, not an existing production database.

1. Architecture and conventions

Recommended approach: modular monolith with PostgreSQL, EF Core migrations, object storage for files, and background workers for document/AI processing. Keep transactional project data relational; store large files outside the database. Use UUID primary keys, UTC timestamps, decimal monetary values, and soft deletion only where justified.

Rule	Recommendation
Primary keys	id uuid PRIMARY KEY; generated server-side or application-side
Tenant isolation	tenant_id uuid NOT NULL on tenant-owned records; tenant-scoped authorization and composite constraints
Money	numeric(18,2) + currency_code char(3); no floating-point currency
Quantities	numeric(18,4) and explicit unit_code; normalize units where required
Concurrency	row_version bigint or xmin strategy; optimistic concurrency on mutable records
Audit	created_at, created_by, updated_at, updated_by; append-only audit_event for sensitive actions
Deletion	RESTRICT for financial history; archive projects, do not cascade-delete approved documents
Naming	snake_case database columns; PascalCase C# entities; plural table names optional but consistent

2. Domain relationship overview

The following is a textual ERD showing principal cardinalities. Detailed foreign keys and constraints follow in the entity catalog.

Parent	Cardinality	Child / relationship
Tenant	1:N	Users, Projects, Suppliers, Documents, CostCodes
Project	1:N	ProjectMembers, Phases, Tasks, Budgets, BOQs, DailyReports, RFQs, PurchaseOrders
Project	1:N	ProjectDocuments, SitePhotos, ProjectRisks
ProjectPhase	1:N	ProjectTasks
BOQ	1:N	BOQItems
Supplier	1:N	SupplierQuotes, PurchaseOrders, SupplierInvoices
RFQ	1:N	RFQItems and SupplierQuotes
SupplierQuote	1:N	SupplierQuoteLines

Parent	Cardinality	Child / relationship
PurchaseOrder	1:N	PurchaseOrderLines, GoodsReceipts, SupplierInvoices
DailyReport	1:N	DailyReportActivities, DailyReportIssues
Document	1:N	DocumentVersions, DocumentChunks, AIExtractionRuns
Budget	1:N	BudgetLines; CostTransactions link to project and cost code
AIJob	1:N	AIJobOutputs; source document optional

3. Identity, tenancy and access control

tenants [MVP]

Top-level customer account and billing boundary.

Column	PostgreSQL type	Rules / meaning
id	uuid PK	Tenant identifier
name	varchar(200)	Legal / display name
status	varchar(30)	trial, active, suspended
default_currency	char(3)	EUR default
created_at	timestamptz	Required

Relationships: 1:N projects, suppliers, tenant_memberships, documents.

users [MVP]

Global authenticated human identities, separate from organization membership.

Column	PostgreSQL type	Rules / meaning
id	uuid PK	Identity principal
email	citext UNIQUE	Normalized login
display_name	varchar(160)	Human-readable
is_active	boolean	Default true

Relationships: M:N tenants via tenant_memberships.

tenant_memberships [MVP]

Maps users to tenant and organizational role.

Column	PostgreSQL type	Rules / meaning
id	uuid PK	Membership key
tenant_id	uuid FK	tenants.id
user_id	uuid FK	users.id
role	varchar(40)	owner, admin, accountant, engineer, viewer
status	varchar(30)	invited, active, disabled

Relationships: UNIQUE(tenant_id,user_id); tenant 1:N memberships; user 1:N memberships.

project_members [MVP]

Project-specific authorization.

Column	PostgreSQL type	Rules / meaning
id	uuid PK	Membership key
tenant_id	uuid FK	Tenant scope
project_id	uuid FK	projects.id
user_id	uuid FK	users.id
project_role	varchar(40)	manager, engineer, procurement, viewer

Relationships: UNIQUE(project_id,user_id); authorization must check tenant and project membership.

4. Project management

projects [MVP]

Construction projects and high-level financial baseline.

Column	PostgreSQL type	Rules / meaning
id	uuid PK	Project ID
tenant_id	uuid FK	Owning company
code	varchar(40)	UNIQUE(tenant_id,code)
name	varchar(200)	Project title
location_text	varchar(300)	Address or locality
planned_start	date	Optional
planned_end	date	Optional
status	varchar(30)	draft, active, on_hold, completed, archived
currency_code	char(3)	EUR
approved_budget	numeric(18,2)	Optional cached approved baseline

Relationships: 1:N phases, tasks, budgets, documents, reports, procurement.

project_phases [MVP]

High-level work breakdown structure.

Column	PostgreSQL type	Rules / meaning
id	uuid PK	Phase ID
tenant_id	uuid FK	Tenant scope
project_id	uuid FK	projects.id
name	varchar(150)	Foundation, structure, finishing
sequence_no	integer	Ordering
planned_start	date	Optional

Column	PostgreSQL type	Rules / meaning
planned_end	date	Optional

Relationships: Project 1:N phases; phase 1:N project_tasks.

project_tasks [MVP]

Work items and schedule dependencies.

Column	PostgreSQL type	Rules / meaning
id	uuid PK	Task ID
tenant_id	uuid FK	Tenant scope
project_id	uuid FK	projects.id
phase_id	uuid FK NULL	project_phases.id
assignee_user_id	uuid FK NULL	users.id
title	varchar(250)	Task
status	varchar(30)	todo, active, blocked, done
progress_pct	numeric(5,2)	0..100
planned_start	date	Optional
planned_end	date	Optional
actual_end	date	Optional

Relationships: Tasks can have N:M predecessor dependencies via task_dependencies.

task_dependencies [MVP]

Directed task dependency edges.

Column	PostgreSQL type	Rules / meaning
tenant_id	uuid FK	Tenant scope
predecessor_task_id	uuid FK	project_tasks.id
successor_task_id	uuid FK	project_tasks.id
dependency_type	varchar(4)	FS, SS, FF, SF

Relationships: Composite PK predecessor + successor; no self-edge; validate no cycles in service.

5. Estimating, budgets and cost tracking

cost_codes [MVP]

Reusable classification of construction costs.

Column	PostgreSQL type	Rules / meaning
id	uuid PK	Cost code ID
tenant_id	uuid FK	Tenant scope
code	varchar(40)	UNIQUE(tenant_id,code)

Column	PostgreSQL type	Rules / meaning
name	varchar(180)	Concrete, labor, electrical
category	varchar(40)	material, labor, equipment, subcontract

Relationships: Referenced by BOQ lines, budget lines and transactions.

boqs [MVP]

Versioned bills of quantities per project.

Column	PostgreSQL type	Rules / meaning
id	uuid PK	BOQ ID
tenant_id	uuid FK	Tenant scope
project_id	uuid FK	projects.id
version_no	integer	Unique per project
status	varchar(30)	draft, reviewed, approved
source_document_id	uuid FK NULL	documents.id
approved_at	timestamptz NULL	Review completion

Relationships: 1:N boq_items; approval requires engineering review.

boq_items [MVP]

Measured and priced estimate lines.

Column	PostgreSQL type	Rules / meaning
id	uuid PK	BOQ item ID
tenant_id	uuid FK	Tenant scope
boq_id	uuid FK	boqs.id
cost_code_id	uuid FK NULL	cost_codes.id
description	text	Work description
quantity	numeric(18,4)	Nonnegative
unit_code	varchar(20)	m3, m2, kg, item
unit_rate	numeric(18,4)	Currency / unit
line_total	numeric(18,2)	Derived quantity * unit_rate
verification_status	varchar(30)	unverified, reviewed, approved

Relationships: BOQ 1:N items; line_total preferably generated or calculated consistently.

budgets / budget_lines [MVP]

Approved budget revisions and cost code allocations.

Column	PostgreSQL type	Rules / meaning
budgets.id	uuid PK	Budget revision ID
budgets.project_id	uuid FK	projects.id

Column	PostgreSQL type	Rules / meaning
budgets.version_no	integer	Unique per project
budgets.status	varchar(30)	draft, approved, superseded
budget_lines.id	uuid PK	Budget line ID
budget_lines.budget_id	uuid FK	budgets.id
budget_lines.cost_code_id	uuid FK	cost_codes.id
budget_lines.amount	numeric(18,2)	Budgeted amount

Relationships: Budget 1:N lines; only one current approved version per project.

cost_transactions [MVP]

Immutable actual and committed cost postings.

Column	PostgreSQL type	Rules / meaning
id	uuid PK	Posting ID
tenant_id	uuid FK	Tenant scope
project_id	uuid FK	projects.id
cost_code_id	uuid FK	cost_codes.id
source_type	varchar(30)	invoice, payroll, adjustment, commitment
source_id	uuid	Application-validated reference
amount	numeric(18,2)	Signed posting
posting_date	date	Accounting period
status	varchar(20)	draft, posted, reversed

Relationships: Project and cost code 1:N; corrections via reversal postings.

6. Suppliers, RFQs, purchase orders and invoices

suppliers [MVP]

Tenant-specific supplier directory.

Column	PostgreSQL type	Rules / meaning
id	uuid PK	Supplier ID
tenant_id	uuid FK	Tenant scope
name	varchar(200)	Business name
registration_number	varchar(60)	Optional local identifier
email	varchar(254)	Optional
phone	varchar(50)	Optional
is_active	boolean	Active supplier

Relationships: 1:N quotes, orders, invoices.

rfqs / rfq_items [MVP]

Request for quotation and requested quantities.

Column	PostgreSQL type	Rules / meaning
rfqs.id	uuid PK	RFQ ID
rfqs.project_id	uuid FK	projects.id
rfqs.status	varchar(30)	draft, sent, closed
rfqs.due_at	timestamptz	Optional deadline
rfq_items.id	uuid PK	Requested item ID
rfq_items.rfq_id	uuid FK	rfqs.id
rfq_items.description	text	Specification
rfq_items.quantity	numeric(18,4)	Required quantity
rfq_items.unit_code	varchar(20)	Measurement unit

Relationships: RFQ 1:N requested items; supplier quotes reference RFQ.

supplier_quotes / supplier_quote_lines [MVP]

Comparable offers and normalized line-level pricing.

Column	PostgreSQL type	Rules / meaning
supplier_quotes.id	uuid PK	Quote ID
supplier_quotes.rfq_id	uuid FK	rfqs.id
supplier_quotes.supplier_id	uuid FK	suppliers.id
supplier_quotes.currency_code	char(3)	Currency
supplier_quotes.delivery_days	integer	Optional
supplier_quote_lines.id	uuid PK	Quote line ID
supplier_quote_lines.quote_id	uuid FK	supplier_quotes.id
supplier_quote_lines.rfq_item_id	uuid FK	rfq_items.id
supplier_quote_lines.unit_price	numeric(18,4)	Supplier price
supplier_quote_lines.quantity	numeric(18,4)	Quoted quantity

Relationships: RFQ 1:N quotes; quote 1:N lines. Compare taxes, specifications, delivery and terms.

purchase_orders / purchase_order_lines [MVP]

Approved procurement commitments.

Column	PostgreSQL type	Rules / meaning
purchase_orders.id	uuid PK	PO ID
purchase_orders.project_id	uuid FK	projects.id
purchase_orders.supplier_id	uuid FK	suppliers.id
purchase_orders.quote_id	uuid FK NULL	supplier_quotes.id

Column	PostgreSQL type	Rules / meaning
purchase_orders.status	varchar(30)	draft, approved, sent, closed
purchase_order_lines.id	uuid PK	Line ID
purchase_order_lines.purchase_order_id	uuid FK	purchase_orders.id
purchase_order_lines.quantity	numeric(18,4)	Ordered
purchase_order_lines.unit_price	numeric(18,4)	Agreed price

Relationships: PO 1:N lines, 1:N goods_receipts, 1:N supplier_invoices.

goods_receipts / goods_receipt_lines [MVP]

Actual delivered materials and acceptance records.

Column	PostgreSQL type	Rules / meaning
goods_receipts.id	uuid PK	Receipt ID
goods_receipts.purchase_order_id	uuid FK	purchase_orders.id
goods_receipts.received_at	timestamptz	Delivery time
goods_receipts.received_by	uuid FK	users.id
goods_receipt_lines.id	uuid PK	Line ID
goods_receipt_lines.po_line_id	uuid FK	purchase_order_lines.id
goods_receipt_lines.accepted_qty	numeric(18,4)	Accepted quantity

Relationships: Receipt lines map to order lines; prohibit cumulative overreceipt unless exception approved.

supplier_invoices / supplier_invoice_lines [MVP]

Supplier bills, OCR review and reconciliation.

Column	PostgreSQL type	Rules / meaning
supplier_invoices.id	uuid PK	Invoice ID
supplier_invoices.supplier_id	uuid FK	suppliers.id
supplier_invoices.project_id	uuid FK	projects.id
supplier_invoices.purchase_order_id	uuid FK NULL	purchase_orders.id
supplier_invoices.document_id	uuid FK NULL	documents.id
supplier_invoices.invoice_number	varchar(100)	Duplicate check per supplier
supplier_invoices.total_amount	numeric(18,2)	Gross amount
supplier_invoices.status	varchar(30)	captured, reviewed, approved, posted
supplier_invoice_lines.id	uuid PK	Invoice line ID
supplier_invoice_lines.invoice_id	uuid FK	supplier_invoices.id

Relationships: Invoice 1:N lines; match invoice / PO / receipt before posting.

7. Site reporting and operations

daily_reports [MVP]

Daily site report header, approval and source notes.

Column	PostgreSQL type	Rules / meaning
id	uuid PK	Report ID
tenant_id	uuid FK	Tenant scope
project_id	uuid FK	projects.id
report_date	date	Site-local calendar date
author_user_id	uuid FK	users.id
weather_summary	varchar(250)	Optional
status	varchar(30)	draft, submitted, approved
voice_document_id	uuid FK NULL	documents.id

Relationships: 1:N activities, issues, photos; project+date can have multiple authors.

daily_report_activities [MVP]

Work accomplished and quantities.

Column	PostgreSQL type	Rules / meaning
id	uuid PK	Activity ID
report_id	uuid FK	daily_reports.id
task_id	uuid FK NULL	project_tasks.id
description	text	Activity detail
completed_quantity	numeric(18,4) NULL	Measured work
unit_code	varchar(20) NULL	Unit

Relationships: Report 1:N activities; task reference optional.

daily_report_issues [MVP]

Blockers, delays and safety concerns.

Column	PostgreSQL type	Rules / meaning
id	uuid PK	Issue ID
report_id	uuid FK	daily_reports.id
issue_type	varchar(30)	delay, safety, quality, supply
severity	varchar(20)	low, medium, high
description	text	Issue description
owner_user_id	uuid FK NULL	Responsible person
status	varchar(20)	open, resolved

Relationships: Report 1:N issues; can create a follow-up task.

8. Document intelligence and AI

documents / document_versions [MVP]

Document metadata and immutable uploaded revisions.

Column	PostgreSQL type	Rules / meaning
documents.id	uuid PK	Document ID
documents.tenant_id	uuid FK	Tenant scope
documents.project_id	uuid FK NULL	projects.id
documents.kind	varchar(40)	contract, drawing, invoice, report
documents.title	varchar(255)	Display title
document_versions.id	uuid PK	Version ID
document_versions.document_id	uuid FK	documents.id
document_versions.version_no	integer	Unique per document
document_versions.storage_key	text	Private object storage path
document_versions.sha256	char(64)	Integrity hash
document_versions.mime_type	varchar(120)	Allowed MIME type

Relationships: Document 1:N versions; no raw binary files in relational tables.

document_chunks [MVP]

Searchable authorized text segments.

Column	PostgreSQL type	Rules / meaning
id	uuid PK	Chunk ID
document_version_id	uuid FK	document_versions.id
chunk_index	integer	Stable sequence
page_number	integer NULL	Source page
content	text	Extracted text
embedding	vector(1536) NULL	Example dimension; model dependent

Relationships: Document version 1:N chunks; enforce tenant/project access before retrieval.

ai_jobs / ai_job_outputs [MVP]

Asynchronous OCR, transcription, extraction and summarization.

Column	PostgreSQL type	Rules / meaning
ai_jobs.id	uuid PK	Job ID
ai_jobs.tenant_id	uuid FK	Tenant scope
ai_jobs.project_id	uuid FK NULL	projects.id
ai_jobs.document_version_id	uuid FK NULL	Source version

Column	PostgreSQL type	Rules / meaning
ai_jobs.job_type	varchar(40)	ocr, transcribe, extract, summarize
ai_jobs.status	varchar(20)	queued, running, succeeded, failed
ai_jobs.model_version	varchar(100)	Model/prompt revision
ai_job_outputs.id	uuid PK	Output ID
ai_job_outputs.ai_job_id	uuid FK	ai_jobs.id
ai_job_outputs.output_json	jsonb	Schema-validated result
ai_job_outputs.review_status	varchar(20)	pending, accepted, rejected

Relationships: AI job 1:N outputs; never auto-approve financial or engineering results.

audit_events [MVP]

Append-only compliance and operational change history.

Column	PostgreSQL type	Rules / meaning
id	uuid PK	Audit ID
tenant_id	uuid FK	Tenant scope
actor_user_id	uuid FK NULL	User or system
entity_type	varchar(80)	Affected aggregate
entity_id	uuid	Affected record
action	varchar(80)	approved, changed, exported
occurred_at	timestamptz	UTC event time
details_json	jsonb	Redacted change summary

Relationships: Referenced logically by all high-risk workflows; immutable retention policy.

9. Cross-tenant foreign key safety

A simple FK on project_id is insufficient if tenant_id is independently supplied. Prefer UNIQUE(tenant_id,id) on tenant-owned parent tables and composite FKs (tenant_id,project_id) -> projects(tenant_id,id), repeated for relevant references. In EF Core, use composite alternate keys and enforce tenant scope in application services. Never trust tenant_id from an arbitrary client request.

10. Key indexes and constraints

Table	Index / constraint
projects	UNIQUE (tenant_id, code); INDEX (tenant_id, status)
project_tasks	INDEX (tenant_id, project_id, status, planned_end)
boqs	UNIQUE (tenant_id, project_id, version_no)
budget_lines	UNIQUE (budget_id, cost_code_id)
supplier_quotes	INDEX (tenant_id, rfq_id, supplier_id)
supplier_invoices	INDEX (tenant_id, supplier_id, invoice_number); investigate duplicate exceptions

Table	Index / constraint
daily_reports	INDEX (tenant_id, project_id, report_date DESC)
document_versions	UNIQUE (document_id, version_no); UNIQUE (document_id, sha256) optional
document_chunks	INDEX (document_version_id, chunk_index); vector ANN index after benchmarking
ai_jobs	INDEX (status, created_at) for workers; tenant scoped query indexes
audit_events	INDEX (tenant_id, occurred_at DESC)

11. Workflow-to-table mapping

Workflow	Tables and transaction boundaries
Create project	tenants -> projects -> project_members -> project_phases
Build estimate	documents -> document_versions -> ai_jobs -> boqs -> boq_items -> approval
Compare suppliers	rfqs -> rfq_items -> supplier_quotes -> supplier_quote_lines
Purchase and receive	purchase_orders -> purchase_order_lines -> goods_receipts -> goods_receipt_lines
Invoice reconciliation	supplier_invoices -> supplier_invoice_lines -> PO/receipt match -> cost_transactions
Daily voice report	documents -> ai_jobs -> daily_reports -> activities/issues -> approval
AI document question	document_versions -> document_chunks -> access-filtered retrieval -> response with citations
Cost dashboard	budgets/budget_lines + posted cost_transactions + PO commitments

12. EF Core mapping example

Example of the tenant-safe relationship principle (illustrative C#):

```
modelBuilder.Entity<Project>().HasAlternateKey(x => new { x.TenantId, x.Id });
modelBuilder.Entity<DailyReport>().HasOne(x => x.Project)
.WithMany().HasForeignKey(x => new { x.TenantId, x.ProjectId })
.HasPrincipalKey(x => new { x.TenantId, x.Id });
```

Use value converters or strongly typed IDs if desired. Define monetary rounding and currency conversion explicitly. Apply restrictive deletion behavior for financial and audit records.

13. MVP versus later extensions

MVP first	Later / optional
Tenants, users, project memberships, projects, phases, tasks	Subcontractor workforce scheduling and timesheets
Documents, versions, AI jobs, daily reports	BIM/IFC model elements and drawing geometry
Suppliers, RFQs, quotes, POs, receipts, invoices	Warehouse inventory, multiple storage locations
Cost codes, budgets, BOQ items, cost transactions	Equipment telemetry and predictive maintenance
Audit events, approvals and notification queue	Advanced risk forecasts and computer vision progress

14. Recommended migration order

001 Identity and tenancy; 002 Projects and membership; 003 Documents and versions; 004 Cost codes, BOQs and budgets; 005 Suppliers and procurement; 006 Site reporting; 007 Cost postings and invoice matching; 008 AI jobs and search chunks; 009 Audit and indexes. Seed reference units, statuses and roles. Add integration tests for cross-tenant access, FK integrity, financial reversal, duplicate invoices and idempotent imports.

15. Decisions to confirm before implementation

Confirm whether subcontractors require their own login, whether one company may own multiple legal entities, how VAT and invoice numbering work in target customers, whether material stock must be tracked, whether budgets are per project or per phase, and whether approvals need multi-stage signatures. These choices affect table boundaries and constraints.

This schema is designed as an implementation blueprint. Before production, complete a full physical schema migration review, data protection review, and performance test with realistic project/document volumes.